

explain more

#### Assumptions

- Your application could be in Java, Python, or Android (Java/Kotlin).
- Sensitive data means passwords, API keys, tokens, credit card numbers, or personal identifiable information (PII).
- You want to educate developers about the risk and show how to fix it.

---

#### Vulnerable Code – Logging Sensitive Data

##### Example 1: Android (Java/Kotlin)

```
java
// VULNERABLE
String apiKey = "sk-1234567890abcdef";
Log.d("DEBUG", "API Key: " + apiKey); // Secret leaked to logcat
```

##### Example 2: Python (Flask/Django)

```
python
VULNERABLE
import logging
logging.basicConfig(level=logging.DEBUG)

def login(username, password):
    logging.info(f"User {username} tried to login with password: {password}")
    password is written to log file or console
```

##### Example 3: Java (Spring Boot / Log4j)

```
java
// VULNERABLE
logger.info("Payment processed for user: {} credit card: {}", userId, creditCardNumber);
```

##### Example 4: Node.js (console.log)

```
javascript
// VULNERABLE
console.log(Token: ${userToken}); // Secret in stdout
```

---

#### Secure Version – How to Fix

##### Fix 1: Never log secrets at all

```
java
```

```
// SECURE - Android
String apiKey = "sk-1234567890abcdef";
Log.d("DEBUG", "API Key retrieved (not logged)"); // No sensitive value
```

Fix 2: Mask sensitive data

```
python
SECURE - Python
def maskvalue(value):
    if len(value) <= 4:
        return ""
    return value[:2] + "" + value[-2:]

logging.info(f"User {username} login attempt - password: {maskvalue(password)}")
example output: "pard"
```

Fix 3: Use a dedicated sanitizer or whitelist

```
java
// SECURE - Log4j with pattern
logger.info("Payment for user {} - card: {}", userId, "---{}".format(last4));
```

Fix 4: Use structured logging and strip sensitive fields

```
python
SECURE - Python with logging extra
import logging
extra = {'user': username, 'action': 'login'}
Never include password in extra
logging.info("Login attempt", extra=extra)
```

Fix 5: Android – Use Log.println only with safe strings

```
kotlin
// SECURE - Kotlin
val safeMessage = "API key configured (${apiKey.take(4)})"
Log.d("SECURE", safeMessage)
```

---

What each fix does

| Technique | Effect                                                                                                        |
|-----------|---------------------------------------------------------------------------------------------------------------|
| Don't log | Completely removes secret from output. Best for secrets that are never needed in logs.                        |
| Masking   | Shows only partial data (e.g., first 2 + last 2 characters); enough for debugging but useless to an attacker. |

| Parameterized logs | Separates data from message; still requires sanitization of parameters. |  
| Structured logging | Explicitly excludes sensitive fields from structured output (JSON, etc.). |  
| Centralized filter | Use a logging filter/middleware to redact known patterns (e.g., Bearer .+). |

---

#### Next Safe Steps

1. Scan your current log files for any leaked secrets (grep for apiKey, password=, token=, Bearer).
2. Add a pre-commit hook or lint rule that forbids logging of variables named password, apiKey, secret, token.
3. Use a logging library with built-in redaction (e.g., Log4j2 with RegexReplacementConverter, Python structlog with processors).
4. Rotate credentials if you find any that were already exposed in logs.
5. Set logging levels to INFO or WARN in production, never DEBUG unless explicitly needed and controlled.

If you want, I can generate a custom secure logging utility class for your specific language/framework. Just tell me which tech stack you use.